Expense Tracker -- Mentoring Guide

A Streamlit project that logs expenses, filters and deletes them, and summarises spending. This guide walks through how the app works, section by section, and closes with exercises to extend it yourself.

1. Streamlit's rerun model

Streamlit apps are not event-driven the way a desktop GUI is. There are no callback functions wired to individual clicks that run in isolation. Instead, **the entire script runs top to bottom every time something changes** -- a button click, typing in a text box, changing a selectbox. Streamlit calls this a "rerun".

This is why the app is written as one flat script rather than a class with methods: on every rerun, every line from `import json` at the top down to the final `st.bar_chart` call runs again, in order.

The practical consequence: any Python variable declared with a plain = is thrown away and recreated from scratch on every rerun. If you want something to survive across reruns -- like the list of expenses the user has entered -- you cannot just keep it in a normal variable. That's what Section 2 is for.

2. st.session_state

`st.session_state` is a dict-like object that Streamlit keeps alive across reruns, for as long as one browser tab stays connected to the app. The app uses it for exactly one thing: holding the in-memory list of expense dictionaries.

```
if "expenses" not in st.session_state:
    st.session_state.expenses = load_expenses()
```

That guard is the key idiom. It reads the saved expenses from disk *once* -- the first time the app starts for this session -- and from then on, every rerun sees the same list already sitting in `st.session_state.expenses` without re-reading the file. Every place that adds or deletes an expense mutates this same list and then calls `save_expenses()` to write it back to disk, so the JSON file and the in-memory list never drift apart.

3. Reading and writing the JSON file

```
def load_expenses():
    if not os.path.exists(DATA_FILE):
        return []
    with open(DATA_FILE, "r", encoding="utf-8") as f:
        return json.load(f)

def save_expenses(expenses):
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(expenses, f, ensure_ascii=False, indent=2)
```

Each expense is a plain dict: `amount`, `category`, `date` (stored as a string, since JSON has no native date type), and `note`. `expenses.json` is just a JSON array of these dicts -- open it in a text editor after running the app once to see it for yourself.

4. Adding an expense: st.form

The add-expense widgets are wrapped in `with st.form("add_expense_form", clear_on_submit=True):`. Without a form, every widget change (typing a character, moving a selectbox) triggers its own rerun immediately -- fine for a checkbox, but annoying for a multi-field entry like this one, where you want to fill in amount, category, date *and* note before anything happens. A form batches all of that: nothing is submitted until `st.form_submit_button("Add expense")` is clicked, and `clear_on_submit=True` empties the fields afterwards so the form is ready for the next entry.

```
if submitted and amount > 0:
    st.session_state.expenses.append({
        "amount": amount,
        "category": category,
        "date": str(expense_date),
        "note": note.strip(),
    })
    save_expenses(st.session_state.expenses)
```

The `amount > 0` check is a small guard against accidentally submitting an empty/zero entry -- worth noticing as a pattern: validate right before you commit data, not scattered throughout the widget code above it.

5. Filtering and listing

The category filter is an ordinary `st.selectbox` with `"All"` prepended to the category list. Filtering itself is a one-line list comprehension against the full `expenses` list -- nothing is deleted or hidden in `session_state`, only the on-screen `visible` list changes.

The list is then rendered by looping over `enumerate(expenses)` -- the *full* un-filtered list -- and skipping rows not in `visible`. This matters: the loop index `i` is used as the delete button's `key` and to index into `st.session_state.expenses` when deleting, so it has to be the index into the *real* list, not into the filtered one -- looping over `visible` directly would delete the wrong row whenever a filter is active.

6. Deleting: st.rerun()

```
if st.button("■■", key=f"delete_{i}"):
    st.session_state.expenses.pop(i)
    save_expenses(st.session_state.expenses)
    st.rerun()
```

Every delete button needs a unique `key` (here, `f"delete_{i}"`) -- otherwise Streamlit can't tell which of several identical-looking buttons was clicked. After popping the item and saving, `st.rerun()` immediately restarts the script from the top, so the deleted row disappears from the screen right away instead of waiting for the next natural rerun.

7. Summarising: st.metric and st.bar_chart

The total is a one-line `sum()` over a generator expression, shown with `st.metric`. The category breakdown builds a plain dict, `{category: total_spent}`, by hand with a small loop -- and hands that dict straight to `st.bar_chart()`, which knows how to turn a dict, list, or DataFrame into a chart without any extra plotting code.

8. Exercises

- **Edit in place.** Add an "Edit" button next to each row that lets you change the amount/category/note of an existing expense instead of deleting and re-adding it.
- **Date-range filter.** Add a second filter (alongside the category one) using two `st.date_input` widgets for a start and end date, and only show expenses in that range.
- **Monthly totals.** Group expenses by month (from the `date` string) and show a second bar chart of total spend per month.
- **Budget warning.** Let the user set a monthly budget with `st.number_input`, and show `st.warning()` if the current month's total goes over it.
- **CSV export.** Add a `st.download_button` that lets the user download all expenses as a CSV file (hint: build the CSV text yourself with the `csv` module, or just join strings by hand -- no new dependency needed).
- **Multiple currencies.** Add a currency selectbox per expense and make the total handle a mix of currencies sensibly (e.g. show one total per currency instead of summing them together).

Written as a teaching companion to app.py -- read this guide with the code open side by side.